



UNIVERSITÀ
degli STUDI
di CATANIA

REPORT OF THE MACHINE LEARNING PROJECT

ELIA ANTONIO MALVÈ

CDLM PHYSICS LM17

Summary

Problem	2
Dataset	2
Training dataset	2
Test dataset	3
Data labeling.....	4
Methods	4
Learning algorithm	4
CNN Architecture	4
Evaluation	5
Experiments	5
Model choice	5
Hyperparameter selection	5
Demo	9
Code	10
Conclusions	10

Problem

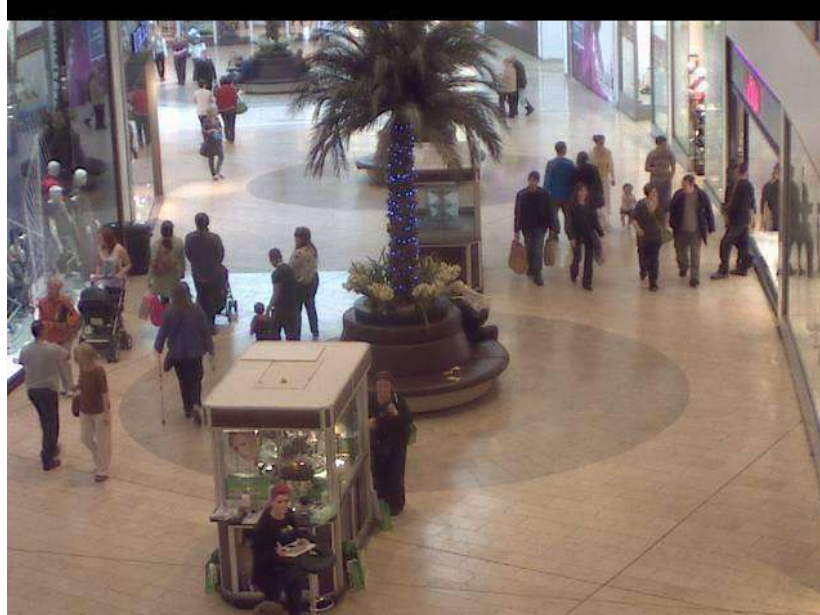
The project aims to develop a model that enables counting the number of people present in an image (or crowd counting) based on a convolutional neural network (CNN). The algorithm is trained on a freely downloadable dataset containing crowd images, each marked with the number of people present. It is then tested on a self-produced dataset. The goal is to estimate the number of individuals in realistic scenarios, by regression. Some possible uses of such software could be related to calculating and optimizing the number of people in a shopping mall, in order to maximize profits, rather than controlling a closed area for security reasons.

security.

Dataset

Training dataset

The dataset used ([accessible here](#)) was built from publicly available resources online, specifically from Kaggle, a site that represents a reference in the data science field. It contains 2000 RGB images at a resolution of 480x640 pixels, which show different frames from the same camera in a shopping mall, as pedestrian traffic varies. Alongside the images, the dataset includes the labels for each image, indicating the actual number of people present.



Sample image of the training dataset.

Each image has been preprocessed (resized to 224x224 pixels) and normalized according to ImageNet standard parameters. Data augmentation techniques (rotations, flips, zoom, color jitter) have also been applied to improve the model's generalization.

Test dataset

The test dataset was built by selecting a set of independent images, not used during training or validation, in order to have a realistic assessment of the model's ability to generalize. Several images were combined: some of them can be freely downloaded from the internet, while others were produced specifically for that purpose. The label is embedded in the filename format, of the type "test_####_%%%", where #### is an incremental id that uniquely identifies the image, and %%% is the label, i.e., the number of people actually present. In total, 50 images were used for testing, 30 of which were taken from the internet and 20 produced by the author specifically for this purpose (with the help of several volunteers). All were chosen under different lighting conditions and viewpoints, so as to evaluate the model's generalization ability. The same resizing was also applied to these, to 224x224 pixels, which is necessary for the input to the CNN used, and they were normalized with the

standard ImageNet parameters, but no type of data augmentation was performed.



Example image of the test dataset, obtained in-house.

Data labeling

Data labels were provided as counts (number of individuals per image). The model was trained to predict a numeric value, aiming to minimize the distance between the prediction and the actual count. The labeling of the test data was done manually. Both datasets are organized into folders, specifically the folder "progetto_ML\dataset\train\" contains the training dataset, in .jpg and .npy format (i.e., NumPy array). In the code, the latter format was used for convenience. Instead, the test dataset has been placed in "progetto_ML\test_dataset\" and consists of images in .jpg format.

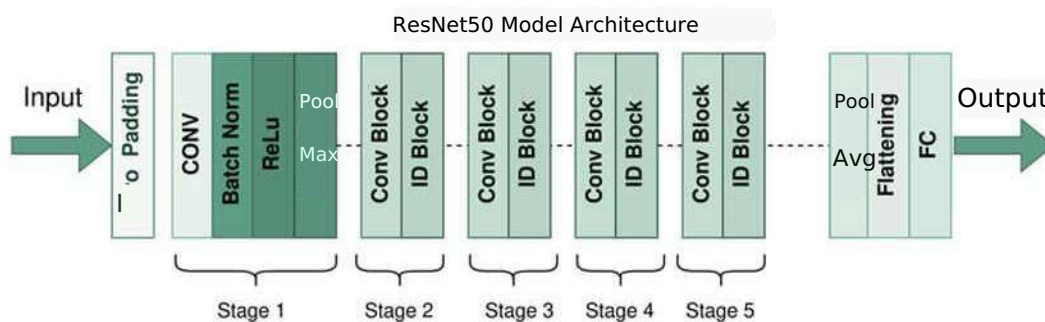
Methods

Learning Algorithm

The operation of the CNN is based on several phases. The first step is image preprocessing, where they were resized to 224x224 pixels and normalized with ImageNet parameters, so they can be input to a CNN, specifically ResNet50. For the training and validation dataset, data augmentation was also performed, as described earlier. A transfer learning strategy was adopted, using the pre-trained ResNet50 as backbone, and dividing the entire dataset into a training set (80%) and a validation set (20%). Optimization was performed via Stochastic Gradient Descent (SGD) with momentum and a regularization parameter. The training progress is also visible in real time via TensorBoard.

CNN Architecture

The chosen network is ResNet50, a CNN characterized by residual connections (the output of a layer is carried to the next level) which reduce the problems of vanishing gradient and exploding gradient. At the output of the network, a fully connected layer was adapted to predict a single value (the estimated count of people). A dropout (random disabling of some neurons during training) of 55% was applied to reduce overfitting.



Schema of the ResNet50 architecture.

Assessment

During training, the metrics Mean Absolute Error (MAE), Mean Squared Error (MSE), and Root Mean Squared Error (RMSE) were monitored. In particular, MAE was considered the most reliable indicator, as it is physically meaningful since it represents an error in the count. The final model and its hyperparameters were selected based on performance on the validation set.

Experiments

Model selection

Several pre-trained models were tested (SqueezeNet, ResNet18, ResNet34, ResNet50). The first attempt was made with the SqueezeNet architecture, as it is particularly lightweight and already seen during the instructional labs. Nevertheless, the results were not satisfactory, so it was decided to switch to a more complex CNN, such as ResNet. After some attempts in that direction, ResNet50 offered a good trade-off between accuracy and computational cost, also considering the complexity of the subsequent ones (ResNet101, ResNet 152), and therefore it was used in all the subsequent phases.

Hyperparameter selection

Several tests were conducted, mainly varying the learning rate, momentum, regularization parameter, and dropout. In particular, the following attempts were made to identify the best parameter conditions:

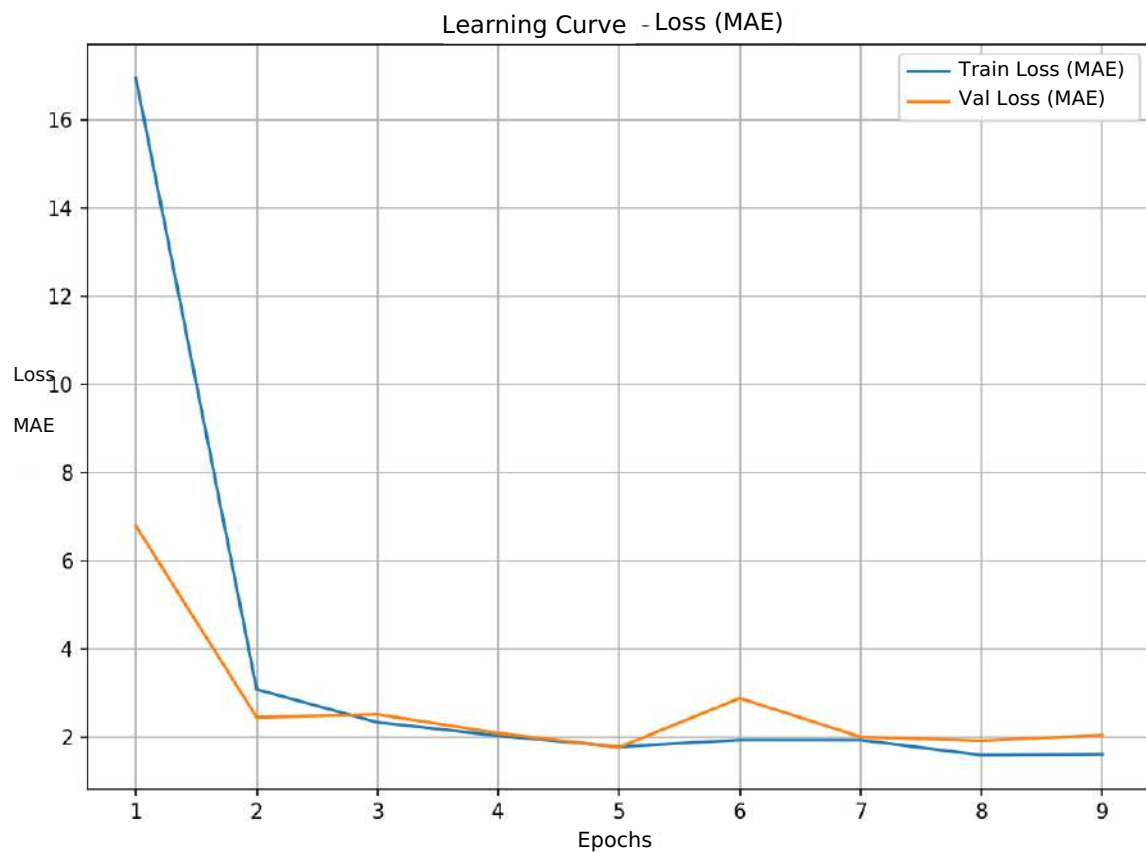
ATTEMPT	LEARNING RATE	MOMENTUM	REGULARIZATION PARAMETER REGULARIZATION	DROPOUT
1	0.001	0.95	0.0001	0.60
2	0.001	0.94	0.0001	0.57
3	0.001	0.93	0.0001	0.58
4	0.0001	0.93	0.0001	0.40
5	0.0005	0.93	0.0001	0.45
6	0.001	0.93	0.0001	0.60
7	0.001	0.92	0.0001	0.55

Having obtained the best parameters in configuration 7, they were fixed, proceeding to vary only one of them at a time for fine calibration. In particular, the momentum was varied in the interval [0.90;0.99], and the dropout in the interval [0.50;0.60], the latter in order to prevent overfitting as effectively as possible. Dynamic reduction of the learning rate was also implemented to avoid local minima of the loss function if there is no appreciable gain for a certain number of epochs.

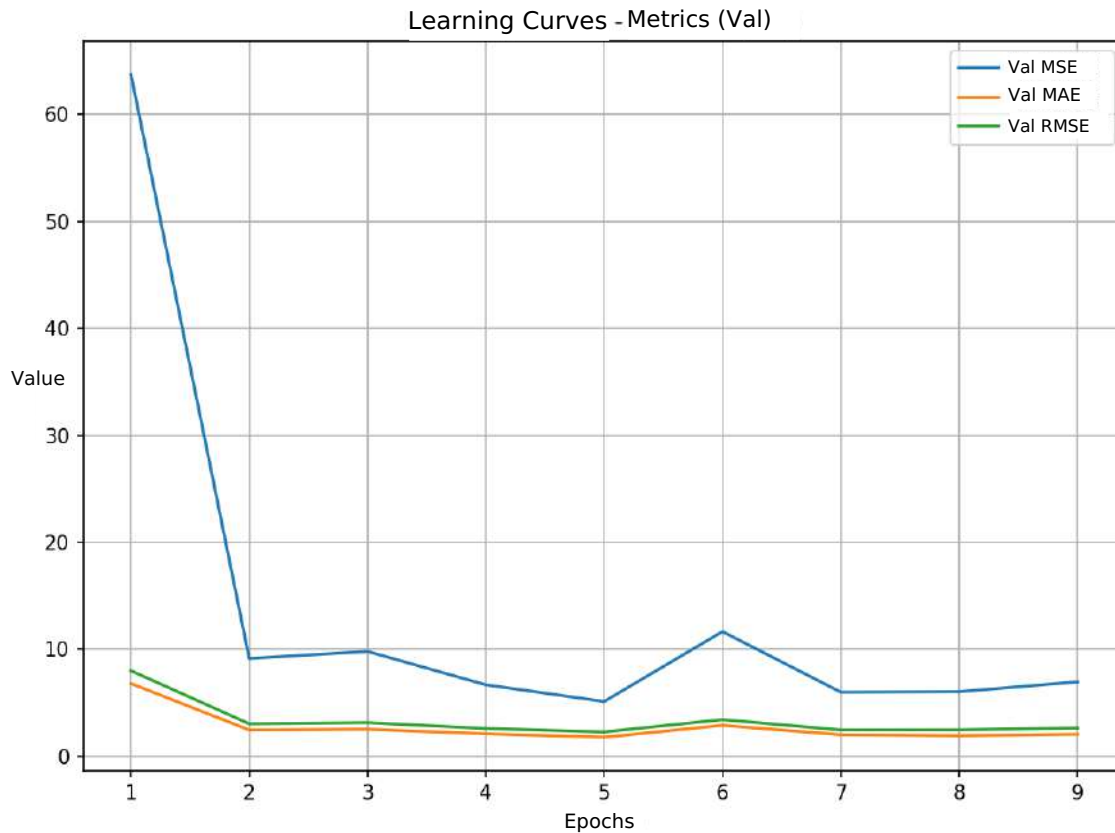
The final hyperparameters were:

- Batch size: 32 (unchanged in all runs, number of samples per SGD)
- Learning rate: 0.001
- Momentum: 0.92
- Weight decay: 0.0001
- Dropout: 0.55
- Patience: 4 epochs (epochs without sufficient improvement to stop the algorithm)

Training stopped after 9 epochs due to early stopping.



Loss curves for the train and test sets as epochs vary. In the first two epochs, the validation loss is lower than the training loss due to dropout, which is present only during training. The trend is completely similar to the expected one.



Trend of metrics as epochs vary.

Final results on the validation

set: - MSE: 5.1072 - MAE: 1.7629

- RMSE: 2.2599

Using MAE as reference, the model makes an average counting error of 1.76 people on the validation set, an acceptable result given the dataset size and the photo configuration in terms of framing.

Test set results: -

MSE: 42.9007 - MAE:

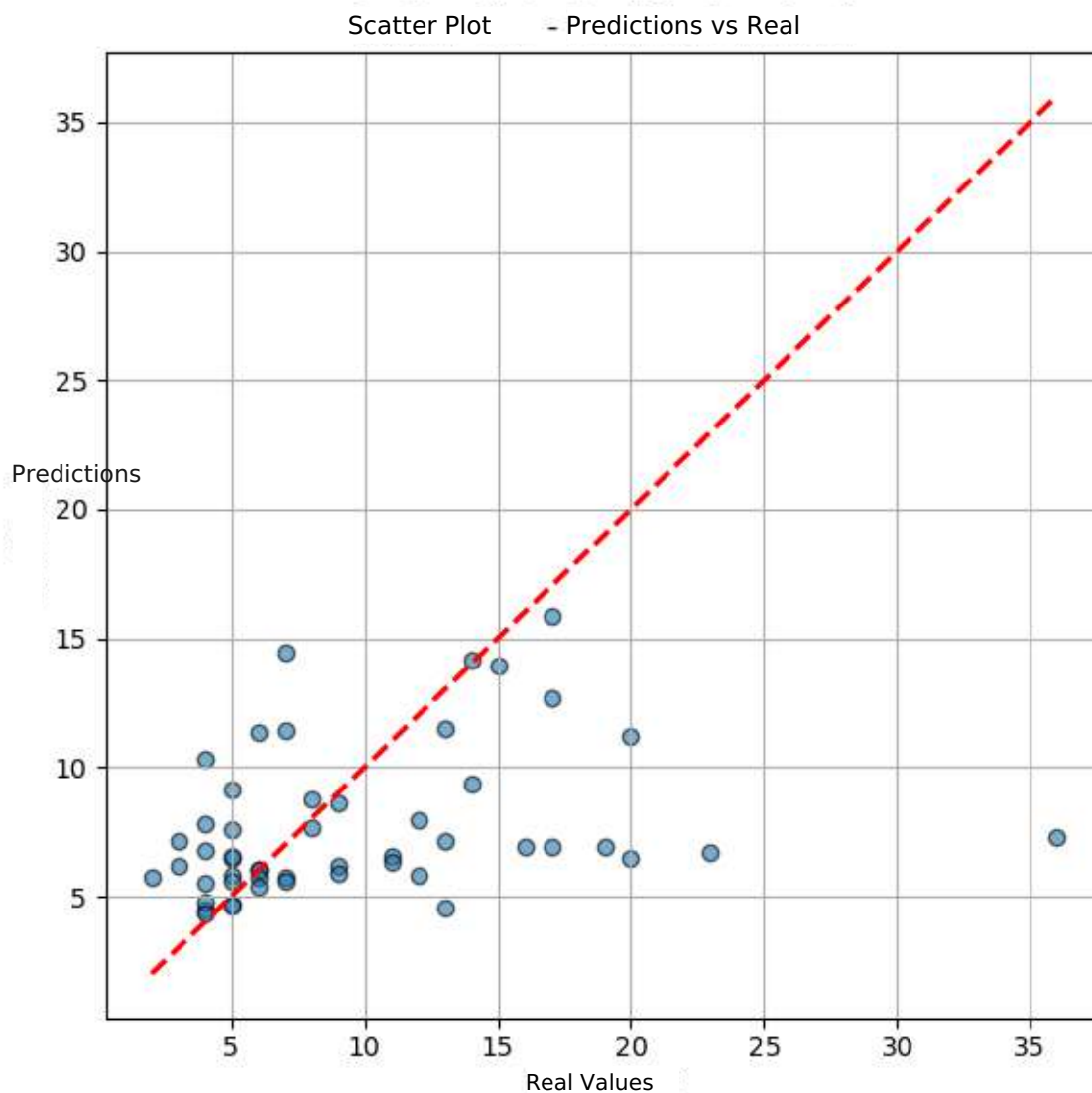
4.1608 - RMSE:

6.5499

- MAE 95% confidence interval: [2.9183, 5.5297] (This parameter was included because, due to the small size of the test set, metric results may fluctuate even with the addition of a few elements. Therefore, this represents the maximum and minimum MAE obtainable, a form of error on the metric).

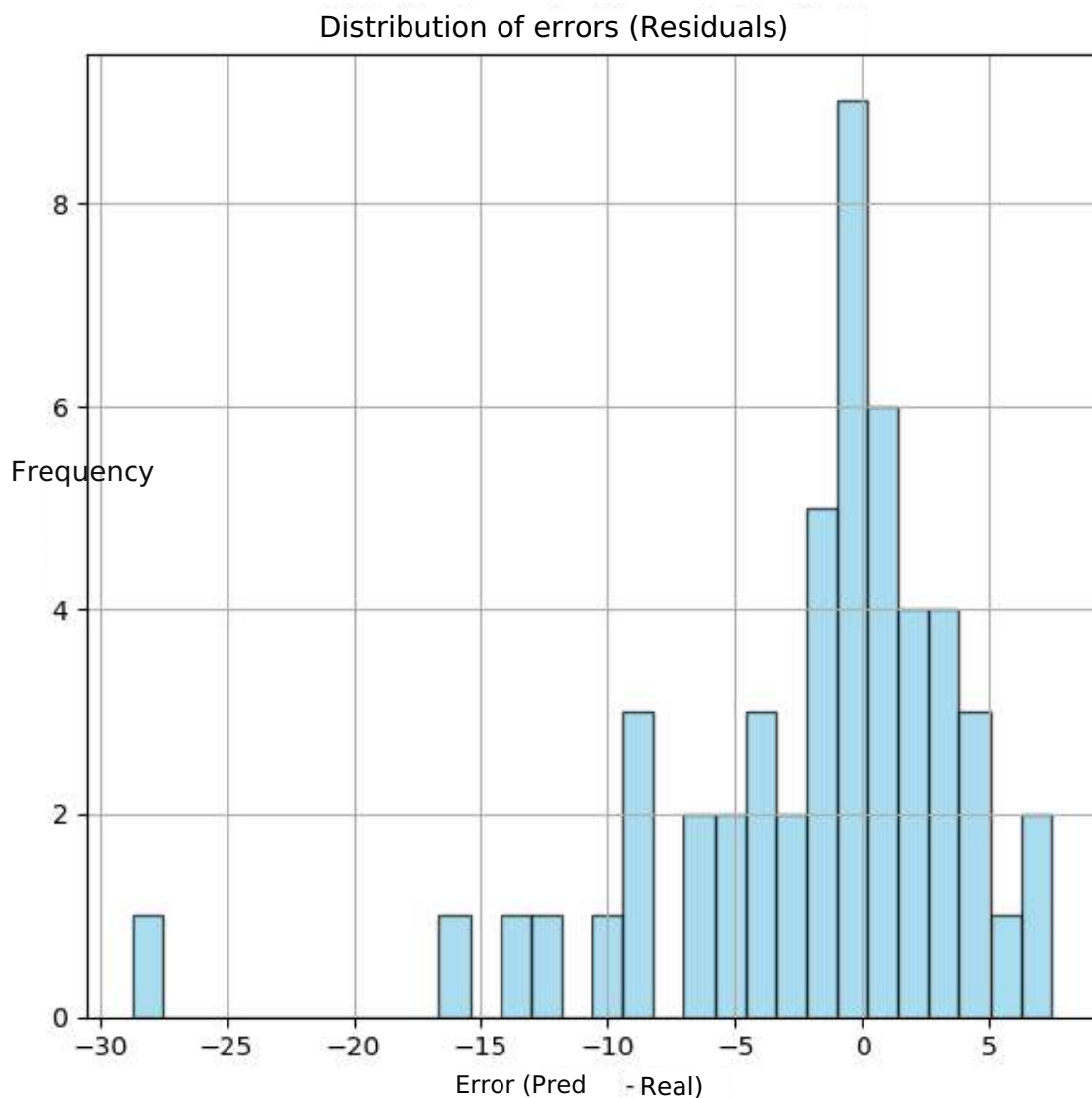
The metrics obtained on the test dataset deviate significantly from the values obtained in training. Several factors contribute to this phenomenon, firstly the rather limited size of the test dataset, which also presents rather different framing, lighting conditions and environment compared to the dataset used for training. In virtue of the emergence of such conditions, the deterioration of metric values is not excessively high, also considering that this is not an overly complex model.

complex.



Scatter plot showing the real count values with the related predictions. The model seems to make better predictions with a low number of people in the image. The points on the bisector are those that have the same number of predictions and real values, therefore the closer a point is to the line in

the question, the better the prediction will have been.



Frequency and distribution of prediction errors, calculated as (predicted value - real value). The peak at value 0 indicates that the model correctly predicts in the majority of cases, despite clearly evident errors, mostly due to overcounting.

Demo

A Python demo has been implemented using the Streamlit library, which allows creating interactive web interfaces, where a zone for image upload has been added and immediately below the prediction of the number of individuals. In particular, the interface allows comparing the estimated value with the real one, with two areas on the same page, one for the prediction and another for the ground truth label, extracted from the file name if it is in the expected format, discussed earlier in the dataset paragraph. If the file name is not in the expected format, an error message is shown, but the predictions are still displayed.

Code

The project is organized in the `project_ML\` folder, containing:

- `code`: the main folder, inside there are three source files: `contapersone_resnet50.py`, which is the code for training the model, `valutazione_test.py`, which is the code to test the model on the test set, and `demo.py`, which is the script for the demo. In addition to these, there is the file `best_ResNet50_crowd.pth` containing the final model. NB. If the training phase is repeated, the model file will be overwritten;

- `dataset`: folder containing the training and validation dataset, it is divided into `train\file.jpg` and `train\file.npy`, which contain the same data but in different formats;

- `relazione`: this folder contains the report of what was done, in `.docx` and `.pdf` format, as well as a demonstrative video of the demo;

- `test_dataset`: Finally, this folder contains the test images.

The instructions for the proper functioning of the codes are present, in the form of comments, in the headers of the codes themselves. These are also commented on every line considered necessary to clarify the function performed.

Anyway, the startup command from the terminal is `python (source file name).py`.

Conclusions

The developed model is able to estimate the number of people present in crowded images, achieving an average MAE of about 4.16 on the test set. Despite good generalization, there are still difficulties in cases where the context or lighting conditions of the test images differ greatly from those of the training set. Future improvements could include using newer and more powerful architectures (such as EfficientNet or Vision Transformers), and/or a larger amount of training data obtained in more varied situations.

This project has been very useful to consolidate what was learned from studying theory and to better understand how a CNN works and its implementation. It also allowed me to engage with the typical problems related to training a neural network

convolutional, such as overfitting, hyperparameter and dataset selection, without forgetting what has been learned in general about the Python language.